

MODULE 2 · CORE WORKLOADS

StatefulSets Basics

Stable identity for pods that aren't interchangeable

Deployment pods are cattle — some pods need to be pets

A Deployment's Pods are interchangeable: any replica can be deleted and replaced by a fresh one with a random name and a clean slate.

Some workloads can't work that way — a clustered database, a leader-election setup, anything where **"pod-0" means something**. A

StatefulSet gives each replica a stable name, stable network identity, and its own storage that follows it.

BY THE END YOU CAN

- Explain ordinal Pod names
- Wire up a headless Service
- Read ordered create / scale-down
- Say why each Pod gets its own PVC

CONCEPT

Stable, ordinal identity — pod-0, pod-1, pod-2

- **Predictable names** — `<statefulset>-<ordinal>`,
e.g. `nginx-stateful-0`
- **Stable DNS** — a headless Service gives each Pod its own address:
`pod.service.ns.svc.cluster.local`
- **Identity survives restarts** — delete the Pod, it comes back with the *same* name, same DNS, same PVC

A Deployment's Pods get a random suffix and no per-Pod DNS — that's the difference this whole lab is about. `clusterIP: None` is what makes a Service "headless".

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-headless
  namespace: training
  labels:
    app: nginx-stateful
spec:
  ports:
    - port: 80
      name: web
  clusterIP: None           # headless — no cluster VIP
  selector:
    app: nginx-stateful
```

Created **before** the StatefulSet — `serviceName` in the StatefulSet spec points at this Service by name.

What actually changes

DEPLOYMENT	STATEFULSET
Pod names — random suffix (<code>web-7d9f8b6d87-x2kp1</code>)	Pod names — ordinal (<code>web-0</code> , <code>web-1</code> , <code>web-2</code>)
Created in parallel, any order	Created sequentially — 0 must be Running before 1 starts
Deleted in any order	Deleted in reverse ordinal — highest first
Shared Service VIP only, no per-Pod DNS	Per-Pod DNS via a headless Service
No per-Pod storage contract	Dedicated PVC per Pod via <code>volumeClaimTemplates</code>

MENTAL MODEL

Reach for a StatefulSet when replicas are *not* interchangeable — each one carries its own identity and data.

Two fields a Deployment doesn't have

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: nginx-stateful
  namespace: training
spec:
  serviceName: nginx-headless # must match headless Svc name
  replicas: 3
  selector:
    matchLabels:
      app: nginx-stateful
  template:
    metadata:
      labels:
        app: nginx-stateful
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
              name: web
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
      volumeClaimTemplates: # stamped once per ordinal
        - metadata:
            name: www
            labels:
              app: nginx-stateful
          spec:
            accessModes:
              - ReadWriteOnce
            resources:
              requests:
                storage: 100Mi
```

serviceName points at the headless Service created first · **volumeClaimTemplates** stamps out one PVC per ordinal — www-nginx-stateful-0, -1, -2 (deeper PVC mechanics: Lab 5.4).

Scaling: imperative to try, declarative to keep

IMPERATIVE — FAST, ONE-OFF

```
kubectl scale statefulset nginx-stateful --replicas=1 -n training
```

Handy for a quick manual test

DECLARATIVE — VERSIONABLE, REPEATABLE

Edit `replicas:` in `statefulset.yaml`, then `kubectl apply -f`

The file stays the source of truth

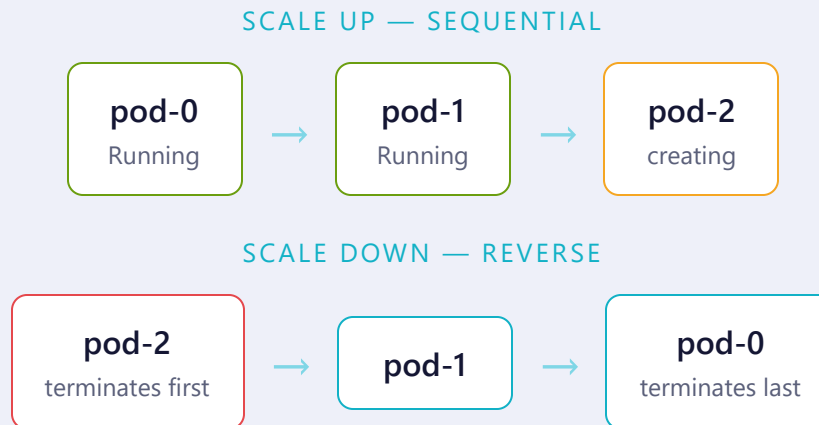
THE BRIDGE TRICK

Preview what a scale would do before committing to it:

```
$ kubectl scale statefulset nginx-stateful --replicas=1 \
  -n training --dry-run=client -o yaml
```

SEE IT

Ordered up, reverse-ordered down



Delete pod-0 directly and it comes back — **same name, same PVC** — because the StatefulSet controller, not you, owns its identity.

Every ordinal owns its own PVC

`volumeClaimTemplates` creates one `PersistentVolumeClaim` (PVC) per replica, named `<template>-<statefulset>-<ordinal>`.

Delete or reschedule a Pod and its PVC is **left alone** — the replacement Pod reattaches to the same PVC, so data survives. Deleting the `StatefulSet` itself doesn't delete the PVCs either; that's a manual step.

This is the "each Pod gets a disk" headline only — Lab 5.4 covers PVCs and `volumeClaimTemplates` in depth.



Where people trip

- **Skipping the headless Service.** It must exist before the StatefulSet, or Pod DNS never comes up.
- **Expecting PVC cleanup for free.** `kubectl delete -f statefulset.yaml` keeps every PVC on purpose.
- **Treating Pods as interchangeable.** pod-0 may hold data the others don't — check before you delete it.
- **Scaling up impatiently.** pod-2 won't even start until pod-1 is Running.

Commands to keep at hand

```
$ kubectl apply -f headless-service.yaml          # create this first
$ kubectl apply -f statefulset.yaml
$ kubectl rollout status statefulset/NAME -n NS    # wait for all ordinals
$ kubectl get pods -l app=NAME --sort-by=.metadata.name
$ kubectl get pvc -n NS                           # one per ordinal
$ kubectl scale statefulset NAME --replicas=N -n NS
$ kubectl delete pod NAME-0 -n NS --force --grace-period=0 # same name comes back
```

Tip: scaling down always removes the highest ordinal first — pod-0 is the last one standing.

RECAP

StatefulSets in one breath

- Ordinal, stable Pod names: pod-0, pod-1, pod-2
- A headless Service (clusterIP: None) gives each Pod stable DNS
- Created 0 → 1 → 2; deleted in reverse, 2 → 1 → 0
- volumeClaimTemplates gives each Pod its own PVC that outlives it

HANDS-ON

Now run Lab 2.6
[labs/2.6/](#)

Next: [3.1 — ConfigMaps](#)